

Simulation of Solar System and Related Experiments

Habiba Nassar

Keywords: Integration Methods, Solar System, Orbital Periods, Planetary Alignment, Energy Conservation

1 INTRODUCTION

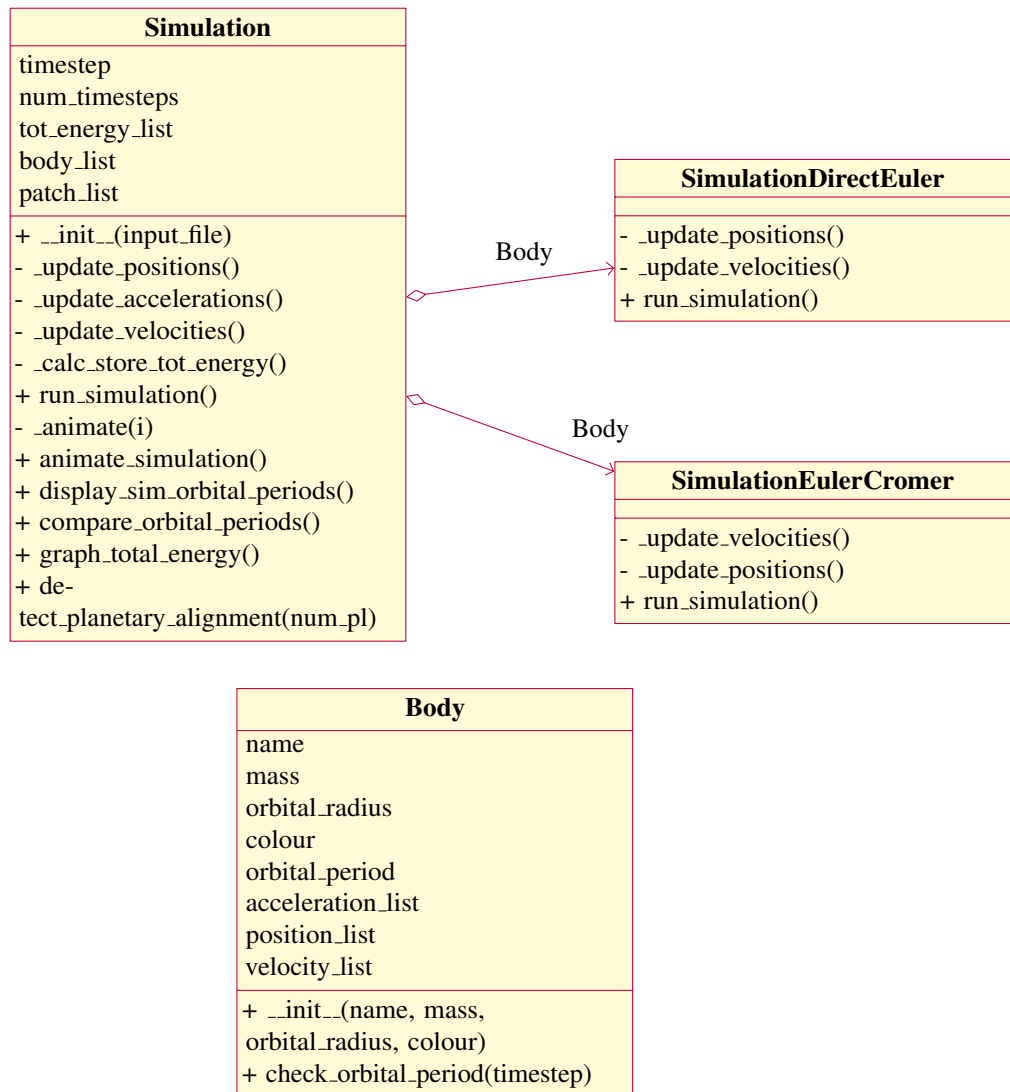
In this simulation, I create an animation of the solar system using Beeman's algorithm. It's known to be very stable and, therefore, provides an accurate representation of the orbital motion of planets. Its stability was evident when compared to other integration methods such as Euler-Cromer and Direct Euler. To compare the precision, I calculated the energy conservation of each method and presented them as graphs (experiment 2). The graph of energy for Beeman was very constant, showing the stability of the Beeman method. This comparison was very beneficial, as it showed the importance of using an integration method that is proven successful when doing calculations that have an impact on worldly things such as space travel, for example.

Another part of this project (experiment 1) was the calculation of orbital periods of planets in my simulation and comparing them to ones provided by NASA. Since NASA is a well-known and trustworthy organisation, its data will be accurate. By comparing my calculated orbital periods to theirs, I can determine the accuracy of my results. For example, if my orbital periods are largely different from theirs, then data obtained from my simulation will be inaccurate and useless, making the simulation a failure.

The final part of my simulation was finding timesteps at which a planetary alignment occurred (experiment 4). This is useful as it shows how frequent they are and can therefore be used to detect when an alignment is likely to take place.

2 METHODS

2.1 UML Diagram



2.2 Main Simulation (Section 3)

The main part of the program – consisting of animating simulation, displaying orbital periods, and calculating/storing total energy – was done using the Beeman method. It is known to be very accurate, making my simulation more precise.

To calculate the positions throughout the timesteps, I had to decide between either calculating them in the **Body** class for every body or calculating them all at once in **Simulation**. Initially, it seemed an obvious choice to keep the updating of positions (as well as velocities and accelerations) in the **Body** class as it is related to the bodies, however, I later realised that would be inefficient for many reasons. Firstly, Experiment 2 would've required creating two more classes that inherit **Body** so that I can update data differently; Secondly, I would've had to pass in the `bodies_list` every time for all the bodies, which would've been unnecessarily complicated as this could lead to issues with updating `bodies_list` incorrectly.

When displaying the orbital periods, I ensured better encapsulation by putting the method for calculating the orbital periods in the **Body** class. This leaves `display_sim_orbital_periods()` to just have to calculate the period for all bodies (using a loop) and print them.

I wrote the code for calculating and writing energy to file in a separate function (instead of in `run_simulation()`) to improve modularisation and make the project easier to code, as I was able to focus on only one part at a time. This was also better in aligning with the project task because each part in the list

of tasks was coded separately instead of one big function.

2.3 Experiment 1: Orbital Periods

For this experiment, I used the method `compare_orbital_periods()` to compare the NASA values with the simulation ones. I put the NASA values in the form of a dictionary for better flexibility, as this allows us to compare the periods even if the planets aren't in the same order. The main calculation used the `check_orbital_period(timestep)`, allowing me to reuse that code for multiple tasks.

Logic behind `check_orbital_period(timestep)`: I looped for the number of timesteps and only stopped when half an orbit was reached. To detect a full orbit I used the following formula: $\text{atan2}(y_i, x_i)$ to calculate the current angle and a variation of it (by replacing i with $i-1$) to calculate the previous angle. Then, I used those to calculate the absolute value of subtracting current angle by previous angle. If this result was more than π , then I knew a half orbit had occurred and could multiply the i value by 2 to get the timestep at which a full orbit occurred. To get the orbital period (in years), I multiplied that by the timestep.

2.4 Experiment 2: Energy Conservation and Alternative Integration Methods

When solving this experiment I used inheritance to make the project code organised and improve maintainability. I created subclasses for the different integration methods and used inheritance to allow them to share the same attributes and methods as the `Simulation` class. This meant I didn't have to rewrite the code for the constructor or any other methods which ensures no code is repeated. The only methods I needed to override were `_update_positions()`, `_update_velocities()` and `run_simulation()`. While `run_simulation()` is largely the same, it had to be overridden because the order in which data is updated differs for each method.

2.5 Experiment 4: Planetary Alignment

There were a few decisions I had to make when solving this experiment. For example, the level of tolerance required to consider the planets in alignment was not given in the task and so I had to choose one that was neither too small nor too big. It is extremely rare to have the planets in true alignment and so having a tolerance of 0 would have been unrealistic. However, a large tolerance would have impacted the accuracy of the experiment. It therefore seemed reasonable to have tolerance of 5° . Additionally, I used relative positions (to the sun) to ensure better accuracy of my results.

3 RESULTS

The following results have all been obtained from my program. This was the input used when program was run:

- timestep - 0.01 (years)
- number of timesteps - 50000
- data for the inner 5 planets as well as sun (all in Earth years and Earth masses)
- number of planets - 4 (overriding the default 5 for planetary alignment)

3.1 Main Simulation (Section 3)

The first result was the **animation** which can be run using the source code but a few snippets of it are also provided below:

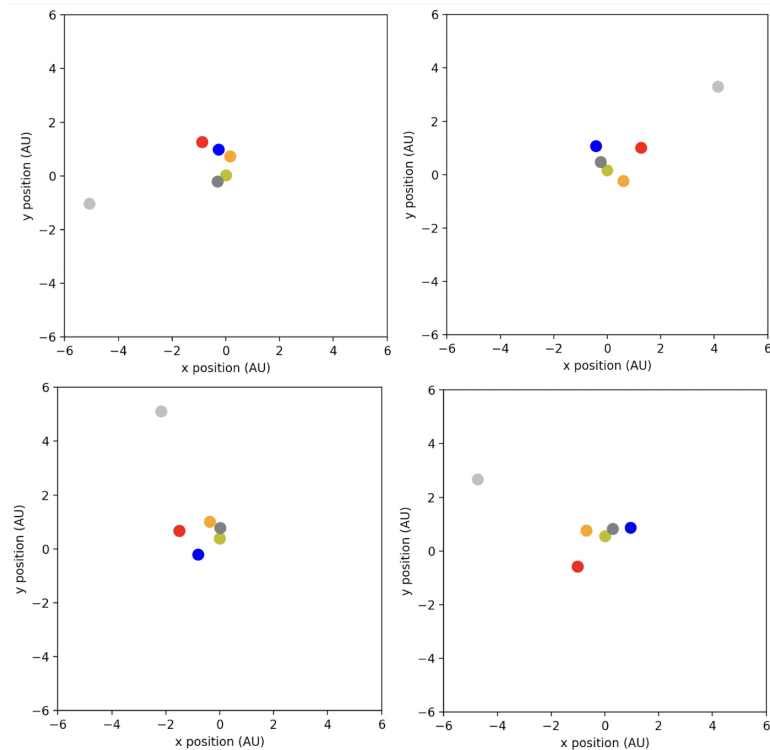


Figure 1. Snippets of solar system animation

The other requirements of the Section 3 task were displaying the simulation orbital periods and calculating/storing the total energy. The total energy values can be found in '*system.energy.txt*' and so will not be provided here. Below are the **simulation orbital periods** in a table format:

Planet	Period (years)	Timestep
Mercury	0.26	26
Venus	0.62	62
Earth	1.02	102
Mars	1.9	190
Jupiter	11.86	1186

Table 1. Orbital periods of planets in earth years

3.2 Experiment 1: Orbital Periods

This experiment compares the simulation and actual orbital periods. Below is a table comparing the orbital periods with a percentage difference:

Planet	Simulation Period (years)	Actual Period (years)	Percentage Difference (%)
Mercury	0.26	0.24	7.88
Venus	0.62	0.62	0.81
Earth	1.02	1.0	2.0
Mars	1.9	1.88	1.06
Jupiter	11.86	11.9	0.34

Table 2. Comparison of simulation and actual orbital periods

3.3 Experiment 2: Energy Conservation and Alternative Integration Methods

The purpose of this experiment is to show the difference in conservation of energy when using different integration methods. To visualise this, I did a graph for each method which are provided below:

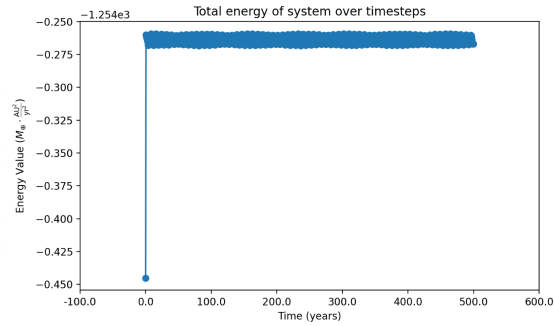


Figure 2. Beeman integration method energy graph

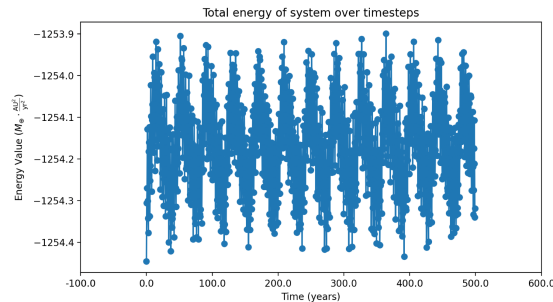


Figure 3. Euler-Cromer method energy graph

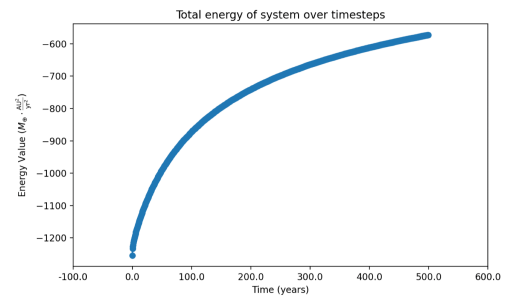


Figure 4. Direct Euler method energy graph

Figure 5. Graphs of total energy for different integration methods

3.4 Experiment 4: Planetary Alignment

The final experiment was to find the timesteps at which planetary alignment occurred. The results below are the alignments for the 4 inner planets:

Timestep
0
1925
47708
49633

Table 3. Occurrences of planetary alignments

4 DISCUSSION

4.1 Main Simulation (Section 3)

The animation of the solar system (Figure 1) was very reliable as I actively worked on ensuring so. When calculating the positions of bodies throughout the timesteps, I considered the sun a moving body rather than a stationary one, despite how small that movement may be. This ensured the relative positions of bodies were always accurate resulting in a more reliable animation.

From the animation, it can be seen that planets closer to the sun orbit it a lot faster than outer ones. This aligns with the law of gravitation as the sun's large gravitational strength is more impactful on bodies near it.

The simulation's orbital periods (Table 1) further show that planets closer to the sun orbit it faster (and therefore have smaller orbital periods). For example, Jupiter is much farther away than the sun and

all the other planets and its orbital period is also much larger. However my results are not totally reliable due to the method I used when calculating the periods. I checked to see when a half orbit occurred and multiplied that timestep by 2 instead of finding the timestep of the full orbit, or even better the average of multiple orbits. This leads to unreliable results because one half orbit may take slightly longer than another or the full orbit may occur at an odd timestep in which case I won't be able to record that (since $\times 2$ will always result in an even number).

4.2 Experiment 1: Orbital Periods

The comparison between simulation and actual orbital periods in Table 2 shows the accuracy of my simulation – the closer they are to actual periods the more accurate they are. To compare how close they are in a more understandable format, I found their percentage difference using the following formula:

$$\% \text{ Difference} = \left(\frac{|\text{NASA}_T - \text{Sim}_T|}{\text{NASA}_T} \right) \times 100$$

Most of my results are very reliable as the difference is $\leq 2\%$ which is very small. However, the result for Mercury seems to be quite inaccurate. This is likely due to how I calculate my periods – the orbital period for Mercury is very small and so being off by even just one timestep impacts the result. A key takeaway from these results is that simulation data should always be validated before making conclusions. If we had assumed that the period for Mercury was 0.26, this could've lead to calculation errors if the data was ever used.

4.3 Experiment 2: Energy Conservation and Alternative Integration Methods

The graphs in Figure 5 show the energy conservation for different integration methods. While their general shape aligns with the method, it is important to note that the y values differ for the different graphs.

Figure 2 is very clearly constant and stable. This shows that the results are reliable as the Beeman integration method is known to be very accurate as shown here. The Euler-Cromer method (Figure 3), on the other hand, is more unstable due to the over-simplicity of its equations which leads to jumps in the graph. However, it's still quite reliable because the difference in the jumps is only 0.5. This jumpiness could also be improved by using smaller timesteps. The Direct Euler method (Figure 4) is the least reliable as shown in the graph. Since it doesn't properly account for the conservation of energy in the system, small numerical errors accumulate over time which leads to an unphysical increase in energy. This is what causes the curved shape of the graph.

From the graphs we can see that the accuracy and reliability of an integration method can be determined from the graph of its energy.

4.4 Experiment 4: Planetary Alignment

The timesteps at which planetary alignment occurs are shown in Table 3. These results are not completely reliable as they have a tolerance of 5° and so are not in perfect alignment. However, this is usually the case for any alignment as it is difficult to achieve perfect alignment. Additionally, the number of planets is only 4 because the more planets there are, the smaller the chances of an alignment.

5 CONCLUSION

In conclusion, the analysis of the planets' orbital motions shows the dynamics of the solar system and how everything fits together. The experiments demonstrate the benefits of using a reliable integration method and, more importantly, highlight the importance of data validation before use. This is done by comparing the simulation and actual orbital periods.

The project could be improved by using a more reliable method to calculate orbital periods to ensure accurate results. Additionally, using a fixed axis for the energy graphs would allow for a better visual comparison. Continued scientific research is essential for further development. This project serves as a foundation for understanding orbital mechanics and the importance of data accuracy to scientific research.

6 WORD COUNT

1910 words